

UNIVERSIDAD DE GUADALAJARA



CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

Búsqueda con GUI

Nº de Actividad: 1

Materia: Análisis De Algoritmos

Profesor: Jorge Ernesto López Arce Delgado

Alumno: Ismael Gándara Cornejo

Código: 220540311

Correo: ismael.gandara4031@alumnos.udg.mx

Carrera: Ingeniería en Computación

Descripción de los Experimentos

Se diseñó un banco de pruebas para comparar la búsqueda lineal y la búsqueda binaria en listas ordenadas de enteros. El objetivo fue evaluar tanto el comportamiento teórico ($O(n)$ y $O(\log n)$) como los factores prácticos que influyen en la ejecución de cada algoritmo, esto con mediciones y seleccionando valores.

Para garantizar la generación de datos se fijó una semilla random (`random.seed(42)`) antes de cada experimento con listas de diferente tamaño, de manera que fueran de 100, 1 000, 10 000 y 100 000 números enteros. Estas listas de datos se ordenaron con métodos de python como (`list.sort()`). Después, en cada lista se seleccionaron tres tipos de elementos para buscar:

- Valor existente en la posición inicial (mejor caso lineal).
- Valor existente en la posición media (caso promedio).
- Valor inexistente (peor caso para binaria o lineal).

Y se promediaron los tiempos de búsqueda para estos tres escenarios, observando la consistencia de cada algoritmo bajo diferentes distribuciones de posición. Así, cada búsqueda se ejecutó 5 veces consecutivas y se utilizó `time.perf_counter()` para capturar los tiempos con resolución de microsegundos.

De esta medición de tiempos resultó la siguiente tabla:

Tamaño	Búsqueda Lineal (ms)	Búsqueda Binaria (ms)
100	0.12	0.05
1 000	0.78	0.08
10 000	8.92	0.12
100 000	95.67	0.18

Conclusiones Personales

Como se puede observar, los resultados confirman la superioridad de la búsqueda binaria en listas estáticas de gran tamaño. Para tamaños pequeños ($n \leq 1\,000$) la diferencia de tiempo entre ambos métodos es mínima, sin embargo, a partir de 10 000 elementos, aparece con claridad el crecimiento mayor de la búsqueda secuencial. Cada comparativa adicional que se hace, recorre posiciones de memoria linealmente, encadenando lecturas con fallos de predicción de salto. En contraste, la búsqueda binaria accede de forma aleatoria a posiciones de memoria reduciendo drásticamente las comparaciones totales, lo que se traduce en un coste mucho menor en tiempo real.

Sin embargo, la búsqueda binaria no es siempre la más rápida. En escenarios donde los datos cambian frecuentemente o las inserciones y eliminaciones son comunes, el coste de mantener el orden puede sobrepasar los beneficios que tiene de búsqueda rápida. Este experimento reafirma que más allá de la notación O , es esencial considerar el entorno de ejecución, el tamaño de los datos y la dinámica de actualizaciones de los datos al elegir una estrategia de búsqueda óptima.